

Relatorio de Implantacao

Ambiente Docker e Kubernetes (Minikube)

Trabalho Pratico de Sistemas Distribuidos - 2026/1

Universidade Federal do Espirito Santo - UFES / CCENS

Aplicacao: Catalogo de Produtos (API Gateway - Product Service - PostgreSQL)

Data: 01/07/2026

1. Contexto e Objetivo

Este documento registra os resultados da etapa de containerizacao e implantacao em Kubernetes da aplicacao de catalogo de produtos, referente as Fases 2 e parte da Fase 3 do planejamento de execucao do trabalho pratico. O objetivo desta etapa foi validar que a cadeia API Gateway -> Product Service -> PostgreSQL funciona corretamente, primeiro via Docker Compose e em seguida dentro de um cluster Kubernetes local (Minikube), antes da instalacao do Chaos Mesh e da execucao dos experimentos de caos propriamente ditos.

2. Build e Validacao Local (Docker Compose)

2.1 Imagens construidas

Componente	Imagem base	Resultado
api-gateway	python:3.11-slim	Build OK
product-service	python:3.11-slim	Build OK
postgres	postgres:16-alpine	Pull OK

Observacao: durante os pulls das imagens base ocorreram falhas intermitentes de rede (timeout na autorizacao do registry Docker Hub). O problema nao era do ambiente local, resolveu-se ao repetir o comando (2a ou 3a tentativa).

2.2 Testes end-to-end via docker compose

Com os tres containers em execucao (postgres, product-service, api-gateway), a cadeia completa foi testada via curl diretamente na porta 8000 do gateway:

Operacao	Requisicao	Resultado
Health check gateway	GET /health	200 - circuit_state: closed
Health check service	GET /health (8001)	200 - status: ok
Listagem vazia	GET /products	200 - []
Criacao de produto	POST /products	201 - produto criado com id=1
Listagem apos criacao	GET /products	200 - 1 item retornado
Busca por id	GET /products/1	200 - produto retornado
Busca id inexistente	GET /products/999	404 - not found
Remocao	DELETE /products/1	204 - no content

RESULTADO: cadeia Gateway -> Service -> Postgres validada com sucesso

3. Implantacao no Cluster Kubernetes (Minikube)

3.1 Ambiente

Item	Configuracao
Ferramenta	Minikube v1.38.1
Driver	docker
Recursos alocados	4 CPUs / 6 GB RAM
Addon habilitado	metrics-server (necessario para o HPA)
Imagens carregadas no cluster	api-gateway:latest, product-service:latest (via minikube image load)

3.2 Comandos executados (reprodutibilidade)

```
minikube start --driver=docker --cpus=4 --memory=6g
minikube addons enable metrics-server
minikube image load api-gateway:latest
minikube image load product-service:latest
kubectl apply -f k8s/deployments/ -f k8s/services/
```

3.3 Status dos recursos apos o deploy

Deployment	Replicas prontas	Status
api-gateway	2/2	Running
product-service	2/2	Running (com restarts iniciais - ver secao 4)
postgres	1/1	Running

3.4 HorizontalPodAutoscaler (HPA)

Logo apos o deploy, o HPA reportava o alvo de CPU como "unknown", pois o metrics-server leva cerca de 1 a 2 minutos para coletar a primeira amostra de metricas dos pods. Apos esse periodo, o HPA passou a exibir um valor real:

Momento	TARGETS (cpu atual/alvo)	REPLICAS
Logo apos aplicar os manifests	cpu: <unknown>/50%	2
~2 minutos apos o deploy	cpu: 4%/50%	2

RESULTADO: pipeline de metricas (metrics-server -> HPA) funcional

3.5 Teste end-to-end dentro do cluster

Utilizando kubectl port-forward para o Service api-gateway (NodePort, porta 8000), repetiu-se o teste funcional ja realizado no Docker Compose:

Operacao	Requisicao	Resultado
Health check	GET /health	200 - circuit_state: closed
Criacao de produto	POST /products	201 - produto criado com id=1
Listagem	GET /products	200 - 1 item retornado

RESULTADO: aplicacao 100% funcional dentro do Kubernetes

4. Achado Relevante: CrashLoopBackOff no Boot do Product Service

4.1 Descricao

Durante a subida inicial dos pods, os dois replicas do product-service entraram em CrashLoopBackOff, reiniciando 2 vezes cada, antes de estabilizarem em estado Running.

4.2 Causa raiz

O product-service tenta se conectar ao PostgreSQL assim que o processo sobe (na fase de lifespan do FastAPI, ao criar as tabelas). Como o Deployment do product-service nao possui nenhuma dependencia declarada em relacao ao Deployment do postgres, o Kubernetes agendou os pods de ambos praticamente ao mesmo tempo. O postgres ainda nao estava pronto para aceitar conexoes quando o product-service tentou conectar, causando falha na inicializacao e o restart automatico pela politica padrao do Kubernetes.

4.3 Por que isso importa para o trabalho

Esse e exatamente o tipo de falha que a disciplina pede para observar e mitigar: o sistema nao ficou fora do ar de forma permanente (o Kubernetes se recuperou sozinho via restart), mas o tempo de estabilizacao e maior do que o necessario, e o mesmo padrao de falha pode se repetir apos um PodChaos (Pod Kill) se o pod recriado do product-service subir antes do postgres estar pronto.

4.4 Acao corretiva recomendada (pendente)

- Adicionar logica de retry com backoff na conexao inicial ao banco, no lifespan do product-service.
- Alternativa mais simples: usar um initContainer no Deployment do product-service que aguarda o postgres responder (ex: pg_isready) antes de iniciar o container principal.

STATUS: identificado, correcao ainda nao aplicada

5. Status Atual Frente a Rubrica de Avaliacao

Criterio	Peso	Status
Complexidade da Aplicacao	20%	Feito - 3 componentes, Dockerfiles, deployments com 2 replicas
Configuracao do Caos	25%	Pendente - manifestos prontos, experimentos ainda nao executados
Tolerancia a Falhas	25%	Parcial - circuit breaker/timeout/retry e HPA implementados e validados; efetividade sob ataque real ainda nao demonstrada
Observabilidade	15%	Nao iniciado - Prometheus e Grafana ainda nao instalados
Qualidade do Relatorio	15%	Em andamento - este documento e um registro parcial

6. Proximos Passos

- Corrigir a condicao de corrida entre product-service e postgres no boot (secao 4.4).
- Instalar Prometheus + Grafana via Helm e montar o dashboard minimo (latencia, erros 5xx, CPU/memoria).
- Instalar o Chaos Mesh via Helm no cluster Minikube.
- Executar os 3 experimentos (NetworkChaos, PodChaos, StressChaos) com carga ativa via k6 ou hey.
- Preencher a matriz de analise (estado estavel, hipotese, configuracao do ataque, resultado, acoes corretivas) para cada experimento no relatorio tecnico final.